

Operating Systemen

Week 8



Universiteit Leiden
The Netherlands

Gedeeltelijk gebaseerd op: Silberschatz A., Galvin P., Gagne G. (2019). *Operating System Concepts, 10th edition*. Wiley Publishing.
(Originele slides: <https://www.os-book.com/OS10/slide-dir/>)

Directory Structure & Implementation

- The files are organized by structuring them in directories.
- To store a directory, we simply need to store a list of files & subdirectories.
- Note that directories can be nested.
- This can be done in different ways.
- A simple linear list is easy to implement, read & manage.
- However, search (a common operation) requires linear scans. Does not scale to (very) large directories.

Directory Implementation

➤ To get around performance problems of simple linear lists, we could cache directory contents in memory (VFS tree) and maintain tree-based index.

➤ Although, the use of a different on-disk data structure might be more suited.

➤ Examples:

-Use hash tables to reduce seek time (example: ext4). We need to be able to deal with collisions however.

-Some file systems (example: XFS) implement B+ trees on disk.

Storing File Attributes

- Next to a file's content, we need to store attributes of that file (metadata).
- File attributes are stored in a data structure that is generically referred to as a File Control Block (FCB).
- Note that filename is not included in the FCB, this is because filename and FCB might be stored separately.

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks
Source: Silberschatz et al., Operating System Concepts, 10 th Edition

Storing File Attributes (2)

➤ Where is the FCB stored?

➤ Generally, two options:

-(1) The FCB is stored as part of an entry in a directory's list of files. So in this list the filename and associated FCB are stored.

-(2) The FCB is stored separately in a table.

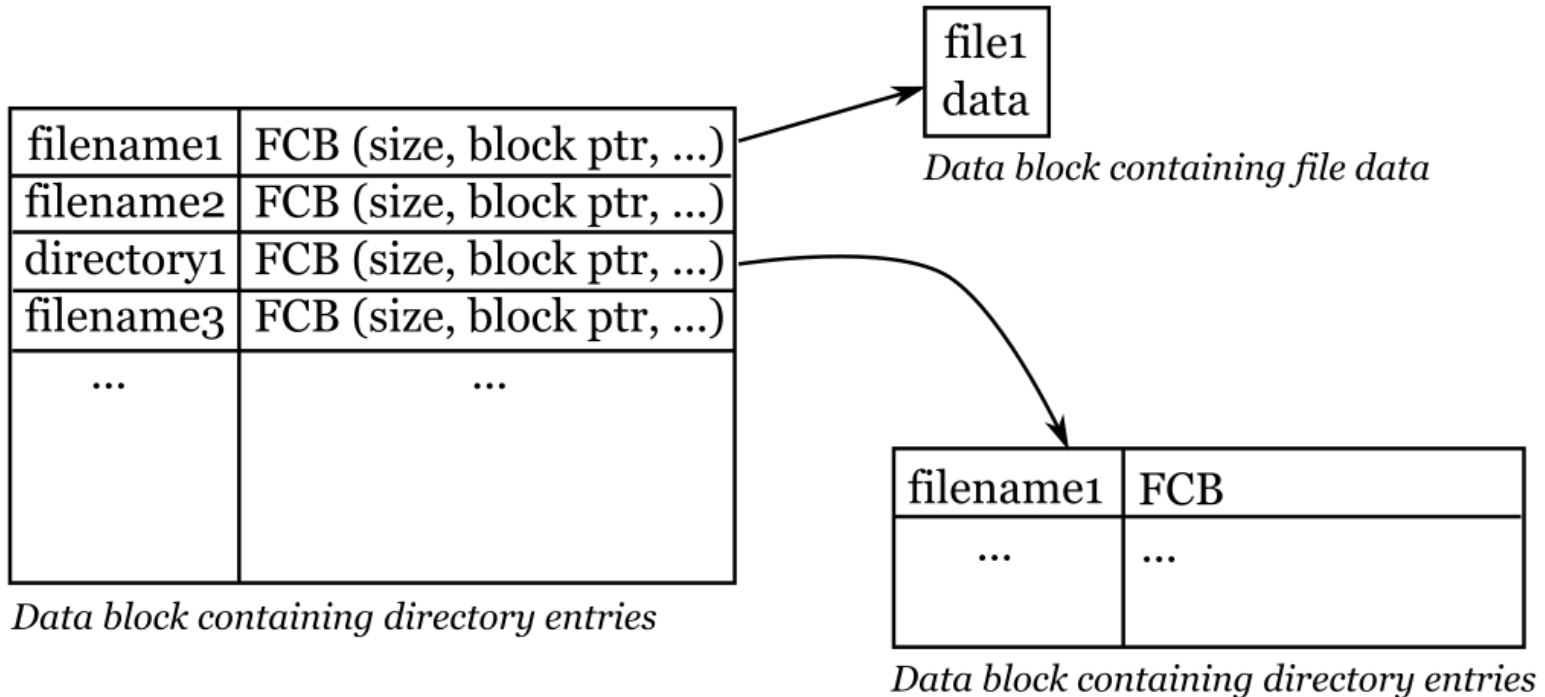
- Each file is identified by a number (often *inode*).

- Directories store pairs of filename and inode. The inode then points at the FCB in the table.

- Multiple filenames (in different directories) may point at the same inode number (and thus at the same attributes & file contents).

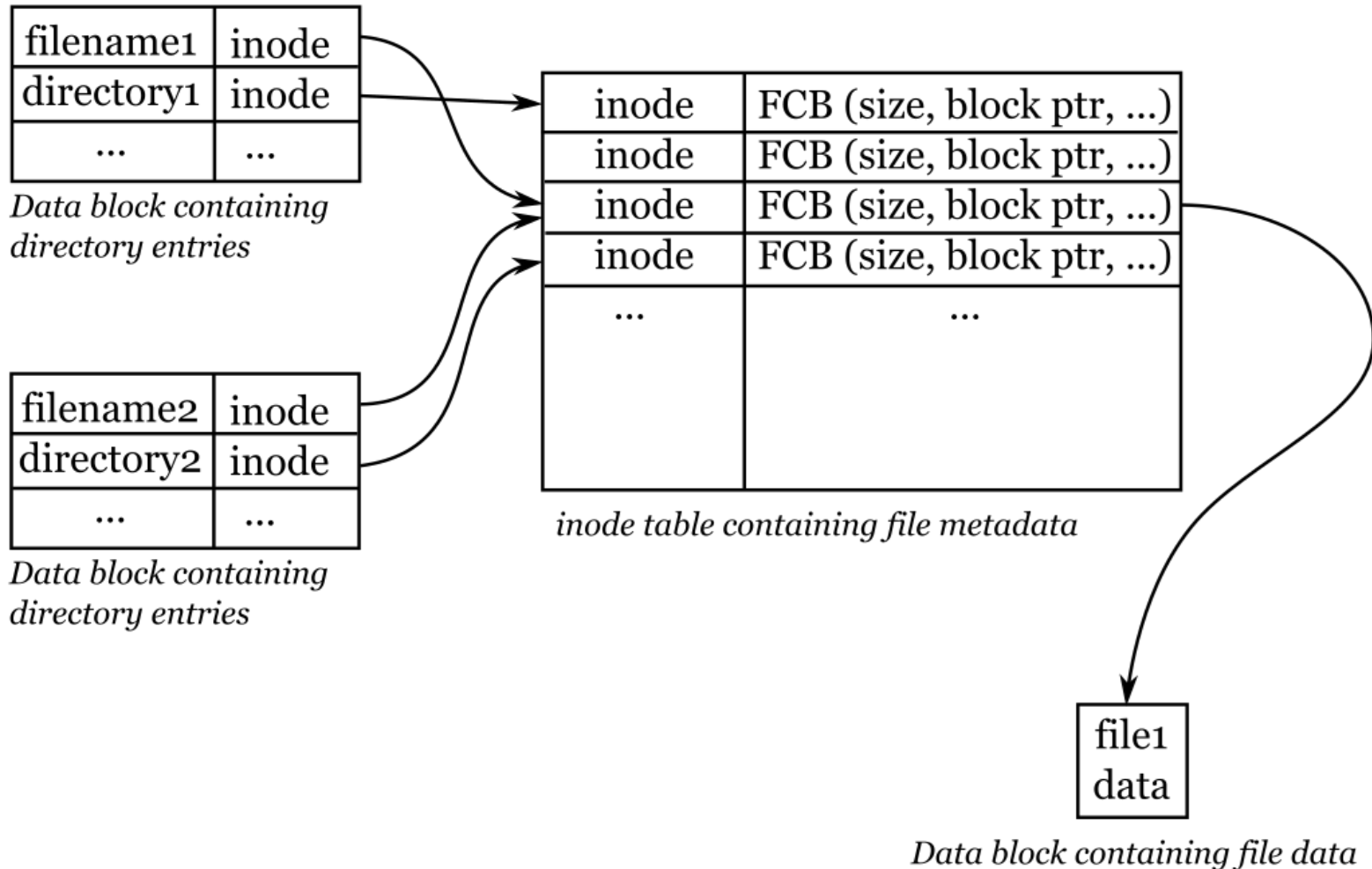
FCB in directory entries

File Control Block (FCB) stored as part of directory entries



FCB in inode table

File Control Block (FCB) stored in a separate table of files, e.g. inode table



Disk Block Allocation

➤ Our next problem concerns the storage of the mapping of the logical address space of a file to physical disk blocks.

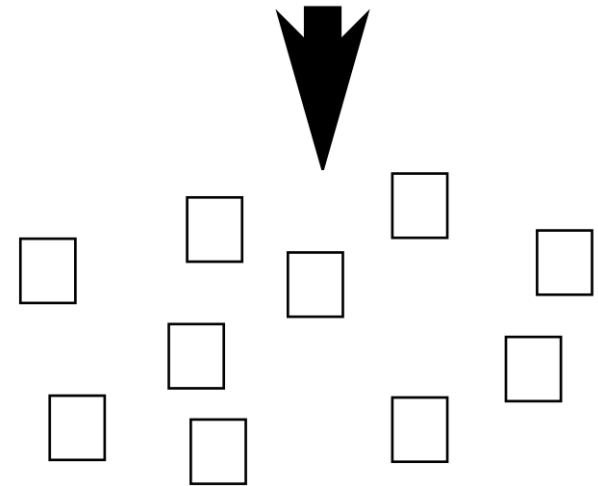
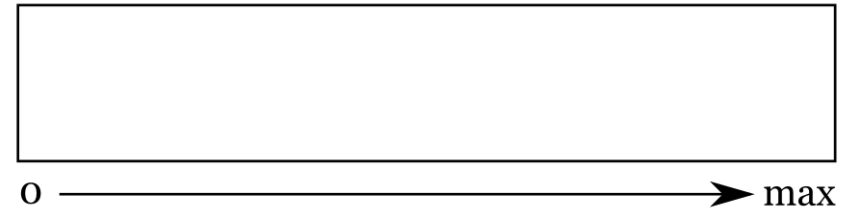
-Physical disk blocks typically 512 or 4096 bytes in size.

➤ The logical address space is contiguous, but the data does not have to be contiguously stored on physical disk.

➤ How do we register what is stored where?

-(We cannot keep this in memory, we must store it on disk as well.)

File: contiguous, logical address space



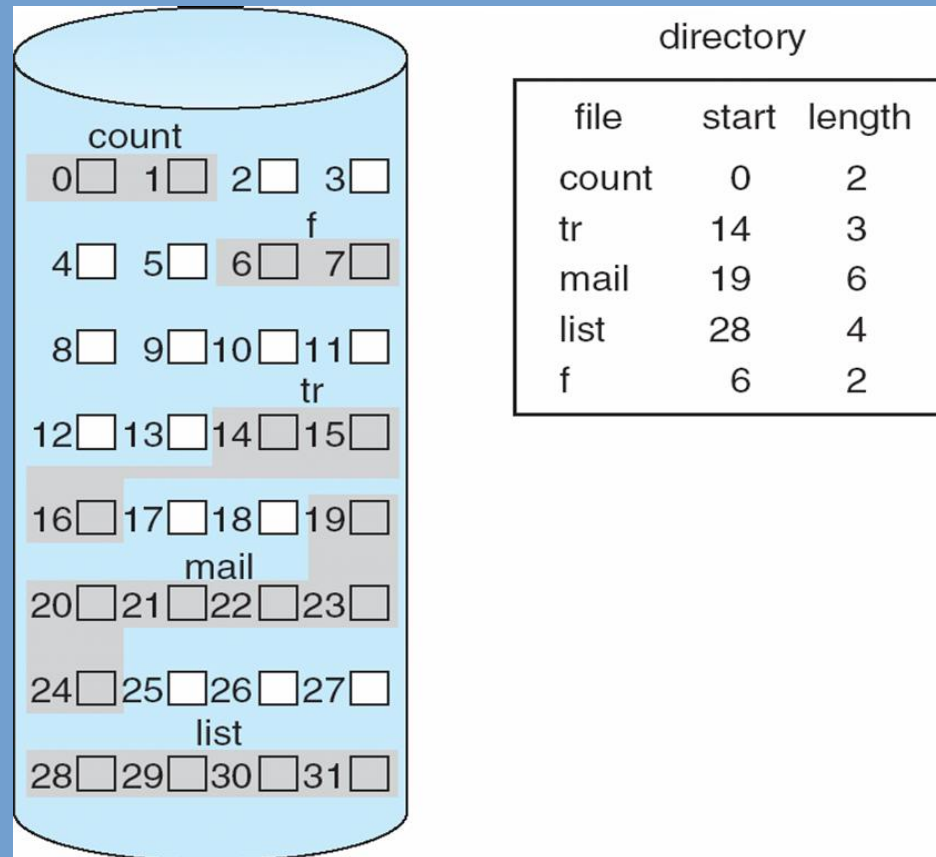
Fixed-size disk blocks

Contiguous Allocation

- Of course, we can choose to require the physical storage to be contiguous.
- In this case, we only have to register start block and size (in number of blocks) in the file's attributes.
- Trivial implementation.
- The problems start to arise when data is appended to files in the future. There may be no space to grow.
- Find another hole that is large enough. Rewrite the entire file there.
- Searching for holes may become time consuming.
- As with contiguous memory allocation: external fragmentation will occur. Compaction is expensive and disk contents need to be moved to other disk areas. Can be done in the background.

Contiguous Allocation

➤ In an image:



Source: Silberschatz et al., Operating System Concepts, 10th Edition

Linked Allocation

- Avoid contiguous allocation of disk areas, instead allocate non-contiguous blocks and stitch them together.
- This eliminates the problem of external fragmentation.
- And finding a block to extend a file is easy: any free block will suffice (when ignoring performance).
- A linked list is used to create a sequence of blocks that contain the contents of the file.
- The file ends at the NULL (or EOF) pointer.
- Sometimes clusters of blocks are used to increase efficiency (less pointer seeks necessary).
- The file system then actually operates on top of “virtual blocks” with a larger block size than the physical blocks.
- This does increase internal fragmentation (cf. paging & page size).

Linked Allocation (2)

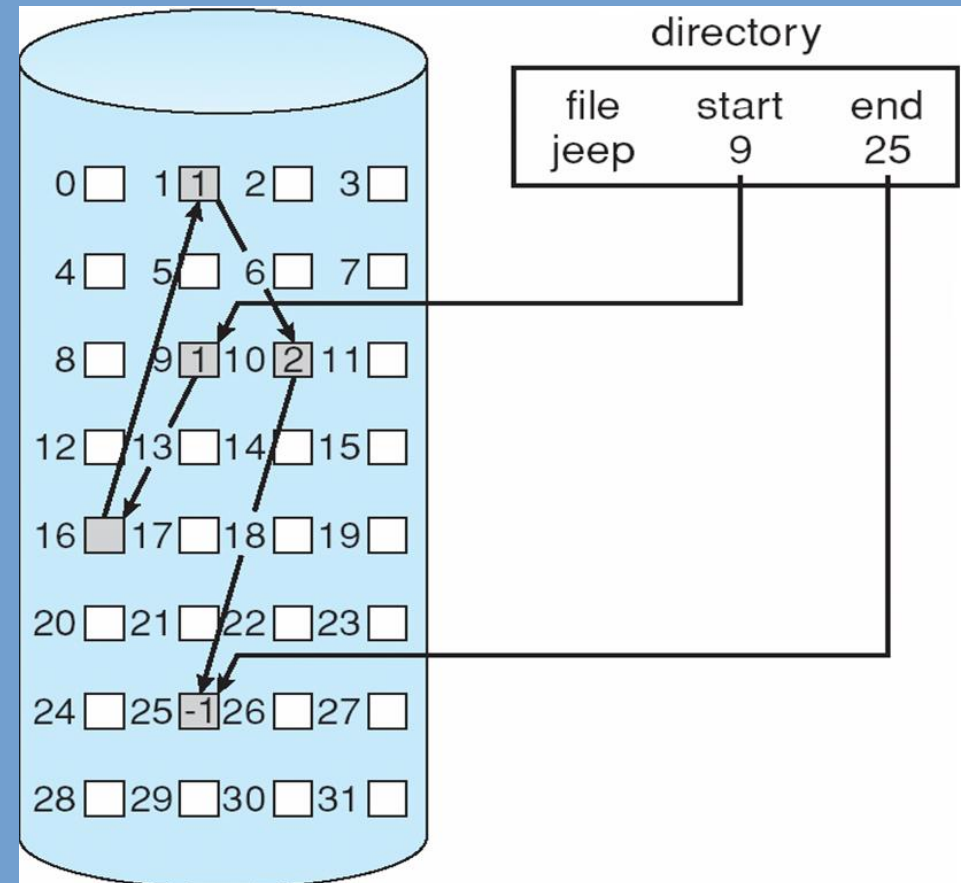
Where to store the pointers to the next block?

➤ This can be done within the blocks themselves, for example in the first or last 4 bytes.

➤ Or, in a separate table (example on next slide).

-FAT (File Allocation Table) model.

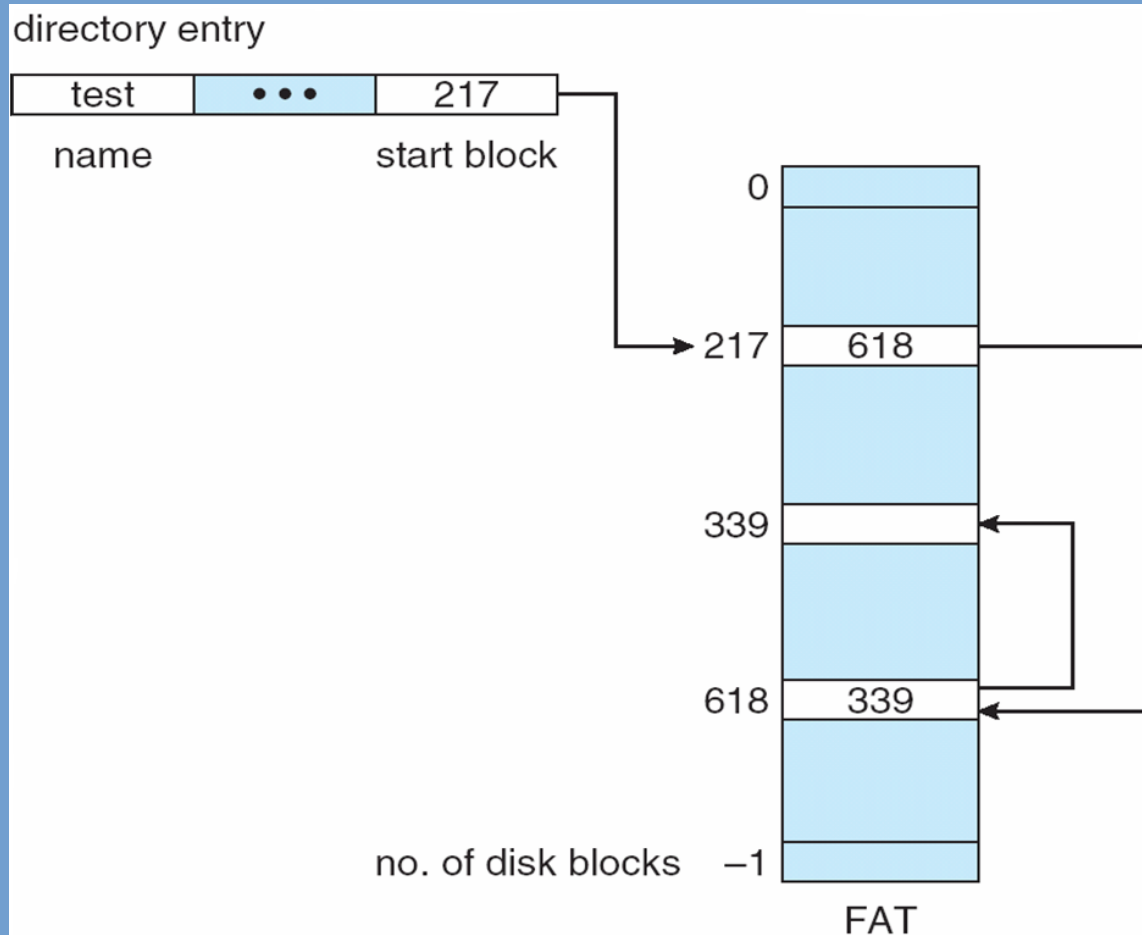
-Big advantage: all pointers are stored on disk contiguously and this area can be easily cached when file system is mounted.



Source: Silberschatz et al., Operating System Concepts, 10th Edition

Linked Allocation (3)

➤ Example of File Allocation Table model:



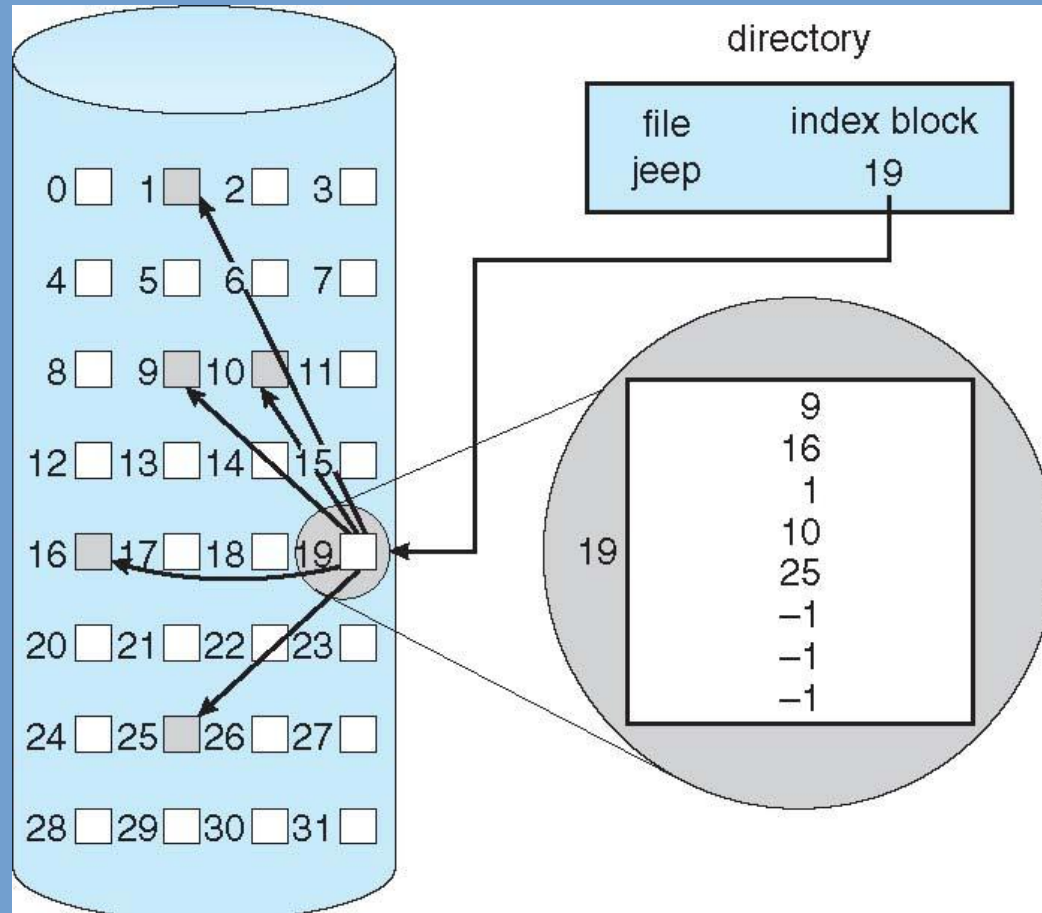
Source: Silberschatz et al., Operating System Concepts, 10th Edition

Indexed Allocation

- A problem that remains with linked allocation is random access. If you want to access the middle of a file, you need to follow all the pointers.
- Indexed allocation tries to alleviate this.
- In this model, each file has one or more *index blocks* that contain pointers to the actual data blocks.
- You could store multiple pointers to index blocks in the FCB.
- You could create a linked list of index blocks (but this results in the random access problem again).

Simple Indexed Allocation

➤ An example of indexed allocation with a single index block:



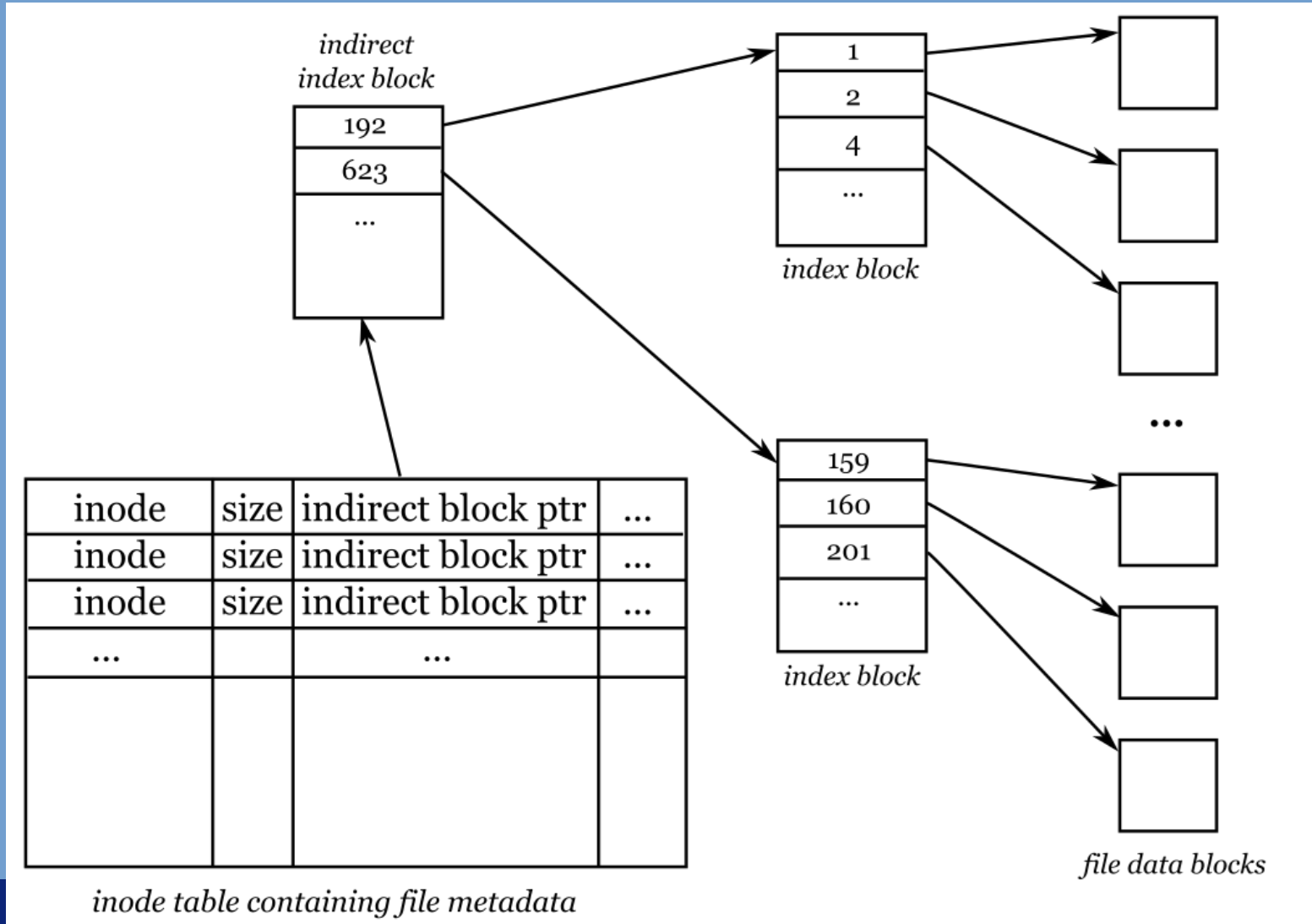
Source: Silberschatz et al., Operating System Concepts, 10th Edition

Extending Indexed Allocation

- We cannot store a long list of index block pointers within the FCB, this would unnecessarily enlarge the FCB (in particular for small files).
- A linked list is also not an option.
- A solution that is in wide-spread use is the multi-level index (again cf. page tables).
 - The first index block contains pointers to other index blocks.
 - In turn, these index blocks contain pointers to data blocks.
 - (Or again to index blocks, the implementor decides the depth of the hierarchy and as such the maximum supported file size).

Extending Indexed Allocation (2)

➤ Schematic of example multi-level index to file contents.

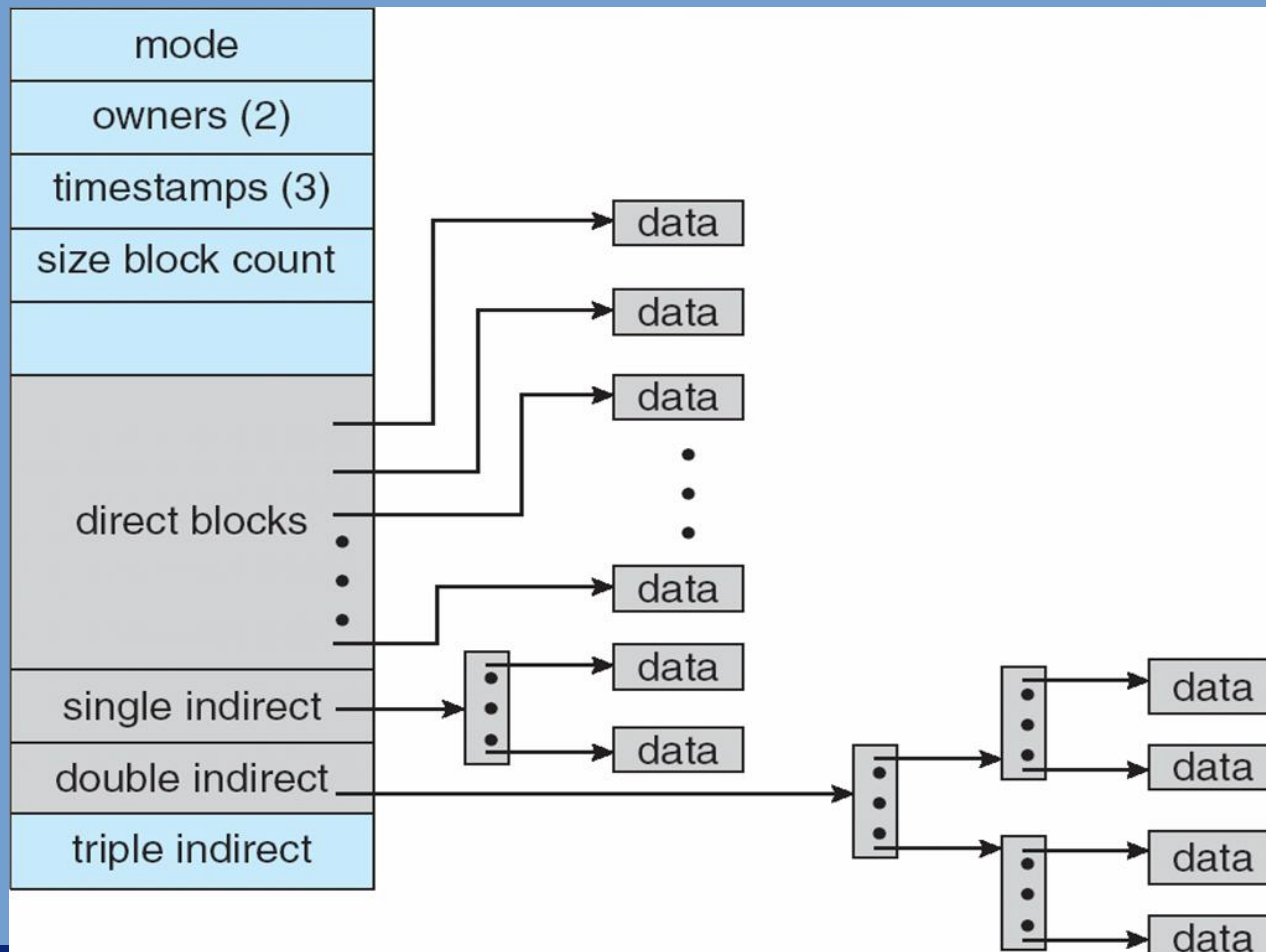


Extending Indexed Allocation (3)

- Disadvantage: also for small files multiple seeks through the hierarchy are needed.
- Solution: Combined Scheme.
- Store different pointers in the FCB:
 - 2-3 pointers to data blocks (*direct blocks*): great for small files.
 - 1 pointer to an index block, which in turn contains pointers to data blocks (*single indirect block*).
 - 1+ pointers to index blocks, which contain pointers to index blocks, which in turn contain pointers to data blocks (*double indirect block*).
 - Could even implement triple indirect block and further ...
 - A picture helps, see next slide.

Extending Indexed Allocation (4)

➤ Example FCB (inode) of UNIX UFS, which uses the combined scheme of indexed allocation:



Performance of Allocation Methods

- Different allocation methods show different performance characteristics.
- We already reasoned that random access performance with linked allocation is problematic.
- Contiguous allocation works well for sequential and random access.
- Less disk seeks.
- A problem with indexed allocation is that it may require multiple disk reads to locate the data block that we are interested in.
- Cf. page table walk.
- The different index blocks may be scattered over the disk.

Free Space Management

➤ On to our next problem: how do we know what blocks of the disk are still available?

-It is clear that scanning 4+ TB disks for “empty” blocks is not an option.

-On the other hand, what is an “empty” block? A block filled with solely zeroes could be allocated to a file!

-We need a separate data structure in which blocks must be marked as taken or available.

• *Free-space list.*

Free Space Management (2)

➤ Two simple schemes:

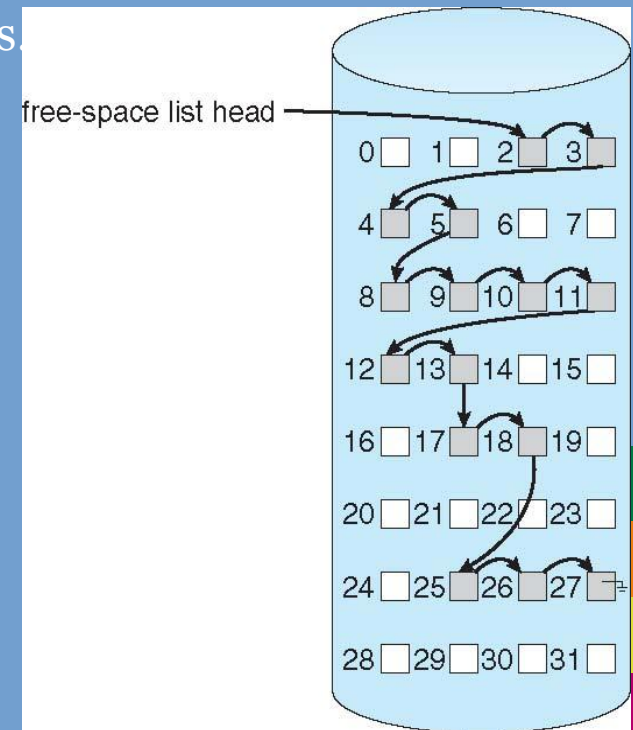
- **bit vector or bitmap.** We store a bitmap on disk that covers all blocks on the disk. 1 or 0 signifies allocated or free.

- For example, Linux ext{2,3,4} file systems use block bitmaps.

- Becomes expensive for large disks with small blocks.

- **Linked list.** Maintain a linked list of free blocks (depicted on the right).

- Expensive to search for sequences of contiguous blocks.



Source: Silberschatz et al., Operating System Concepts, 10th Edition

Free Space Management (3)

➤ Modifications of free-list approach:

-Grouping

- This approach stores the address of the free blocks in the first free block.
- The first free block stores the address of some, say n free blocks.
- Out of these n blocks, the first $n-1$ blocks are actually free and the last block contains the address of next free n blocks.

Free Space Management (4)

➤ Modifications of free-list approach:

-Counting

- This approach stores the address of the first free disk block and a number n of free contiguous disk blocks that follow the first block.

- Every entry in the list would contain:

 - Address of first free disk block.

 - A number n .

[100, 50]

File System Performance

➤ Several design decisions in the implementation of file systems directly influence the file system's performance:

- Disk allocation algorithm that is used
- On-disk data structure for maintaining directories
- How much data is stored in a file entry and the time necessary to maintain this
 - Some systems maintain access time entries.
Expensive! Often disabled.
- Method of allocation metadata data structures.
- Are the data structures used of fixed-size or varying in size?
 - For fixed-size data structures the faster pool allocation can be used.

Disk Subsystem Performance

➤ Some techniques used to improve disk subsystem performance:

- Buffering of disk blocks, keep frequently used disk blocks in main memory. *Buffer cache* or *page cache* (see also later).

- *Asynchronous writes*. Tell the application a write has completed when this is actually not the case. Buffer the written data and write to disk later in time.

 - Some applications do require *synchronous* writes. More expensive (in time), but safer. When the write system call returns, application is guaranteed data was written to disk.

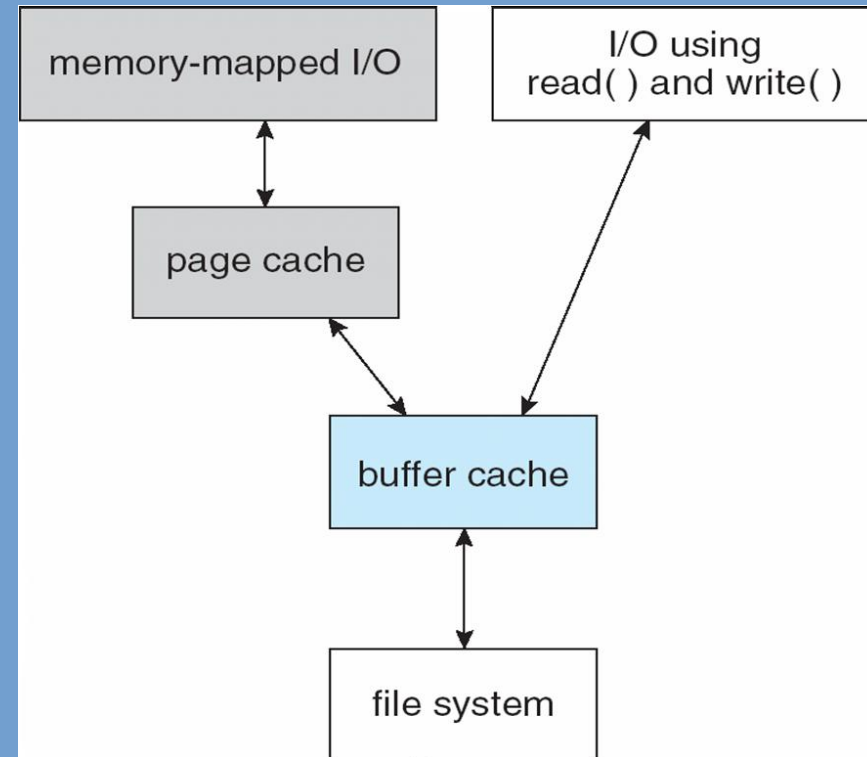
- *Read-ahead*. When OS detects that an application simply reads through a file in sequential order, read next blocks from disk before they are explicitly requested.

 - Similarly, blocks that have been read already can immediately be purged from the buffer cache (*free-behind*).

- What would be faster? Reads or writes? Why?

Page Cache

- Buffer cache was originally based on the caching of individual disk blocks (512 bytes in case).
- Recall our discussion on memory-mapped I/O. Here we need to cache pages (4K or larger) in memory.
- A naive implementation will read a “page of blocks” from disk and cache this data both in buffer cache and in memory-map cache.



Source: Silberschatz et al., Operating System Concepts, 10th Edition

Page Cache (2)

➤ To alleviate this a unified buffer cache was proposed and is now implemented by most systems.

-The buffer cache is eliminated and all disk block caching is done in groups of blocks equal to the page size.

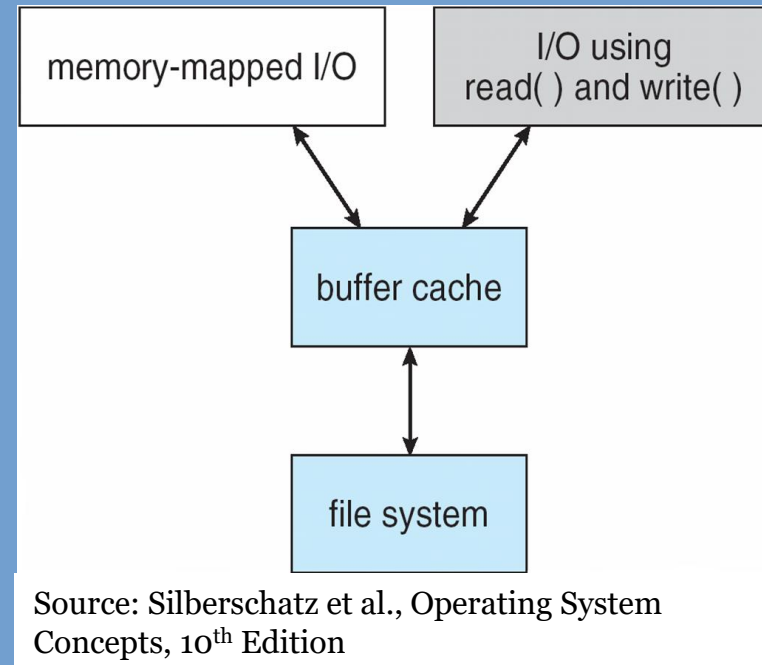
-Ordinary file system I/O is routed through the unified buffer cache.

-Double buffer is eliminated.

➤ Need to be careful with page replacement algorithms.

-High I/O rates may take away pages from ordinary processes to expand the page cache.

-These ordinary processes might now run out of memory and start swapping/paging (I/O load further increases).



File System Corruption

- File systems data structures (or file contents) may be corrupted.
- Bit flip due to broken DRAM, or disk fault.
- Problems cannot be detected in all cases. If you want to detect this for file contents, you need to store checksums of disk blocks within file system metadata (done by ZFS).
- Data structures can be checked by a consistency checker.
- Or file system checker (often abbreviated “fsck”).
- Such a checker visits all file system metadata (traverses entire directory structure) and checks whether this is sound.
- Detected errors can in many cases be fixed.

File System Corruption (2)

- Pitfall of file system checkers: they may fail to repair detected errors.
 - The file system is now left in a partly inconsistent state. Data may be lost!
- How to protect against this?
 - *Back up* data to another storage device, such as magnetic tape, USB drive, external hard drive.
 - Never copy file system one-on-one (so at partition level), this will also copy potentially corrupted file system data structures.
 - So, back ups are not just done to protect against failed hard drives!
 - Given a back up, lost files and/or disks can be restored using a restore or recovery procedure.

File System Corruption (3)

- Another problem of file system checkers: they are REALLY slow.
- It may already take quite some time for 64 GB hard drives, imagine for 4+ TB drives.
- A system crash meant a mandatory file system check to detect and repair inconsistencies: long boot time.
- Early 2000's a solution was proposed: journaling file systems (or log-structured file systems).
- Main idea: maintain a log of modifications done to file system metadata.

File System Corruption (4)

Journaling File Systems:

- Each metadata update becomes a transaction stored in a log file.
- A system call will only return after the log entry is guaranteed on disk.
- At some later point in time, the actual file system data structure will be modified.
- When completed, the entry in the log is marked as processed.
- In case of a system crash, at the next boot the log is checked (“journal replay”).
- All non-processed transactions are now carried out.
- Smaller chance of inconsistencies after crash, no mandatory consistency check on boot after crash so much faster recovery time.
- However, a consistency check is still strongly recommended to be performed once or twice a year.

Chapter 15

- The structure (and name) of Chapter 15 are not very clear.
- We discuss only selected topics from Chapter 15:
 - File systems
 - Disk partitions / volumes / devices
 - File System Mounting
 - Virtual File Systems
 - Remote File Systems (briefly)

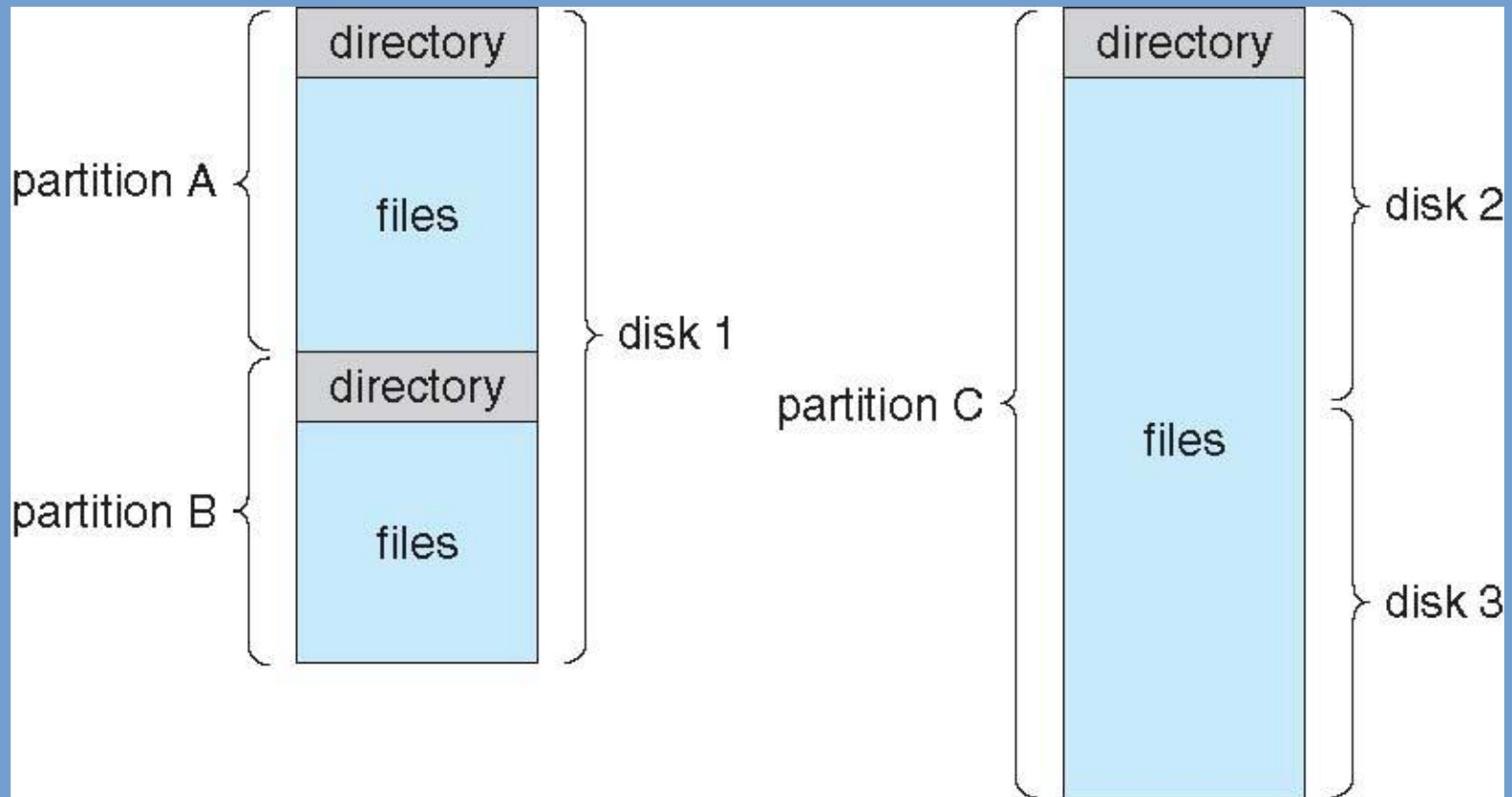
File System

- The file's contents and its metadata need to be stored somehow.
- This can be done in different formats, or as we say using different *file systems*.
 - Well-known file systems: FAT, NTFS, HFS, ext4, xfs, zfs.
 - These are general-purpose file systems. Some systems also implement special-purpose file systems, examples: tmpfs (fast in memory file system), procfs (file system to obtain information about running processes), sysfs.
- The file system also encodes the directory structure.
 - File attributes are sometimes stored as part of the directory structure.
- Attributes of the file system itself are stored in a superblock at a specific location within the file system.

Putting File Systems on Disks

- Next: how / where are file systems stored?
- A file system needs to be stored in a contiguous area on the disk.
- To create these areas, a disk is subdivided into *partitions*.
- Partitions present on a disk are recorded into a *partition table* located at the start of a disk.
- We refer to a partition populated with a file system as a *volume*.
- We can also bundle multiple disks to form a single disk, such that a file system can span multiple disks.

Disk Partitions



Source: Silberschatz et al., Operating System Concepts, 10th Edition

Disk Partitions (2)

➤ Disk partitions are not very flexible.

- Say we want to enlarge the file system in partition A in the example of the previous slide.

- We need a contiguous area, so we would have to move partition B. Expensive operation!

➤ One solution is a logical volume manager, such as Linux LVM.

- Disks are physical volumes, which are placed in a volume group.

- Within a volume group, logical volumes can be created. Logical volumes may span multiple disks and do not have to be contiguously stored on the physical volumes.

- The volume manager is responsible for maintaining the mapping from logical to physical.

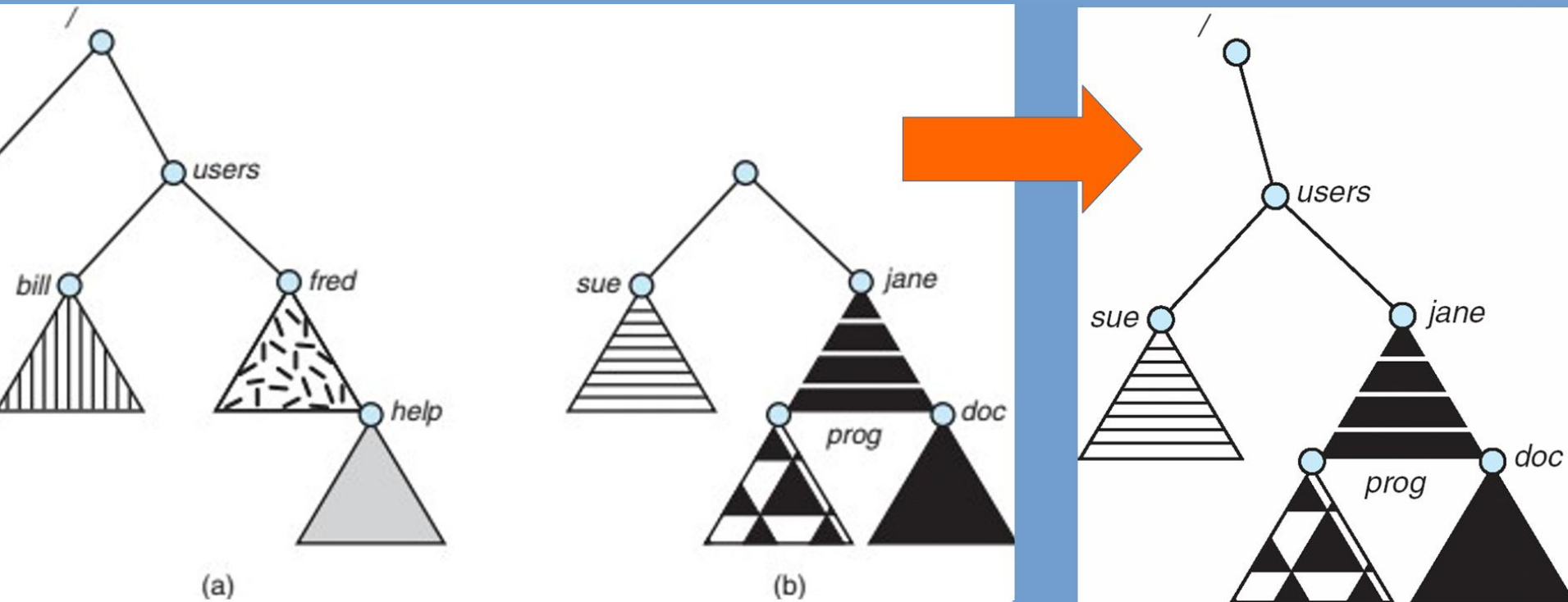
- The logical volume is always contiguous and as such suitable to store a file system on.

File System Mounting

- Assume we have three volumes, how do we access the file systems?
- DOS/Windows solution: drive letters!

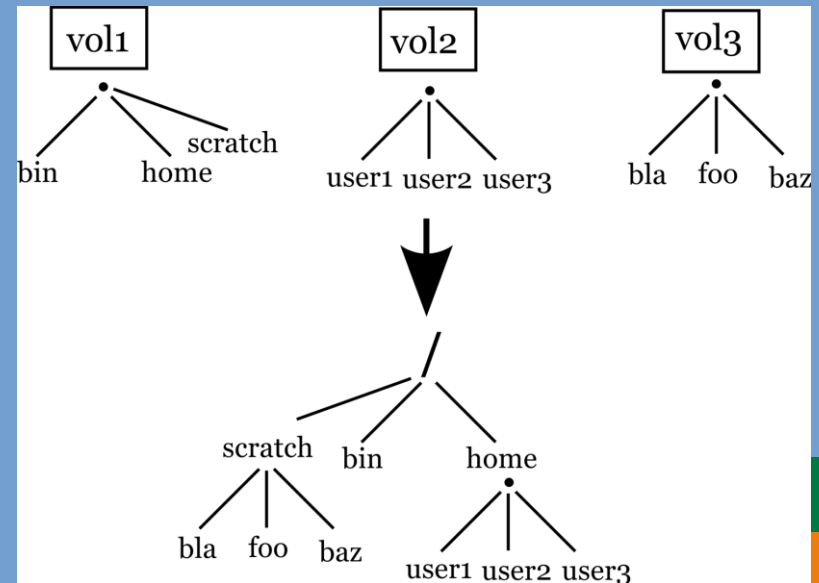
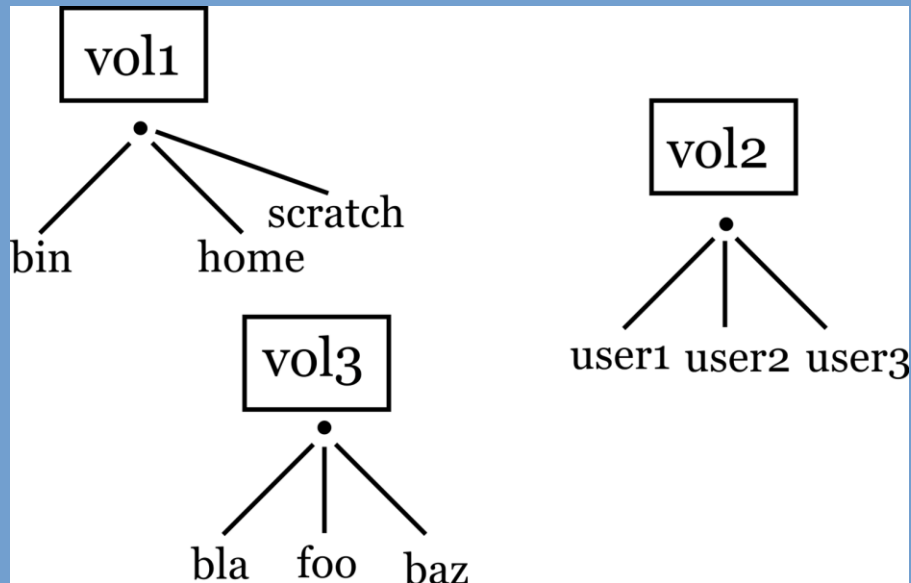
File System Mounting (2)

- UNIX solution: file system *mounting*. We must mount a file system at a *mount point* (typically a placeholder directory) before it can be accessed.
- In the example the file system on the right (b) is mounted at /users. Now this will shadow the content of /users on the left (not always what you want).



File System Mounting

> example:

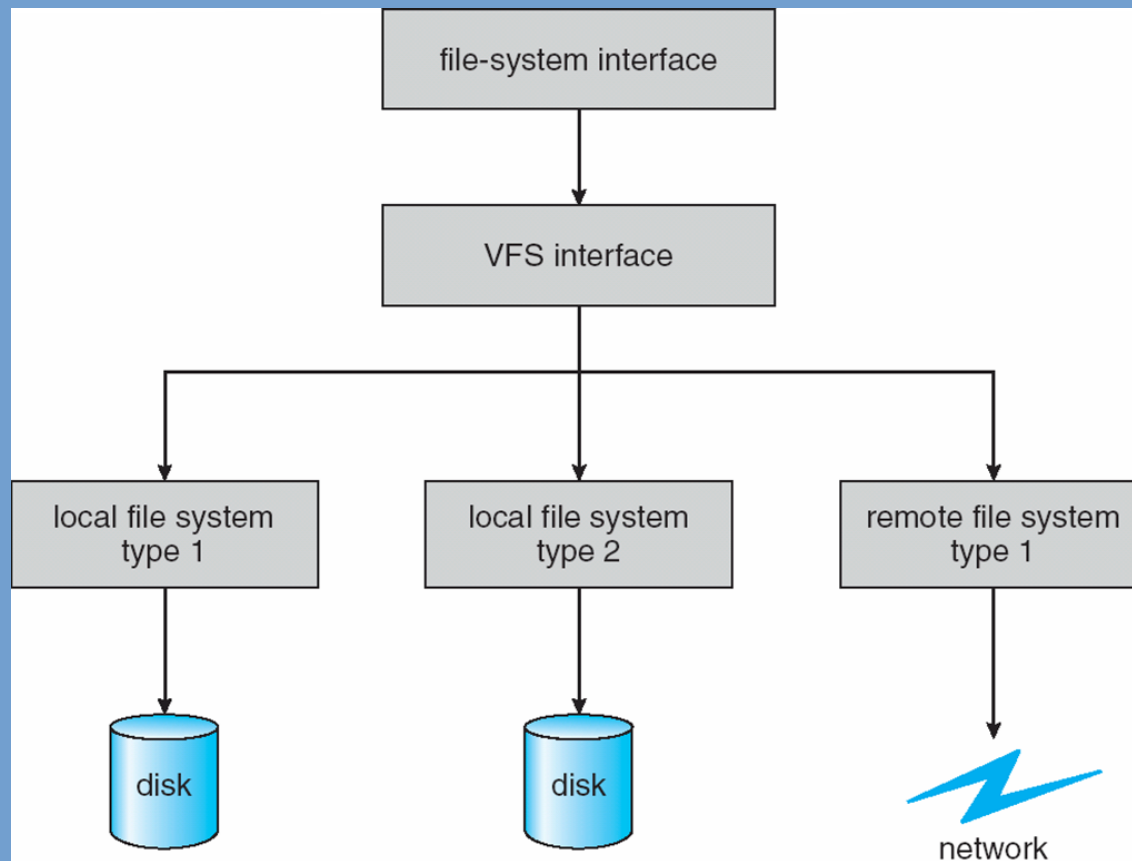


Virtual File System (VFS)

- The Logical File System layer is often structured as a Virtual File System (VFS).
- This is an object-oriented way to implement file systems.
 - For instance, we have a “File” base class that specifies the methods to be implemented.
 - Applications (system calls) are programmed against these generic calls.
 - Different file system implementations subclass this base class and implement the methods specific to that file system.
 - Also consider network file systems here, the file might have to be transferred over the network.
 - We can now have a single tree of files, but the files may be located on different file systems.
 - Through polymorphism the correct functions are called and executed when a particular file is accessed.

Virtual File System (2)

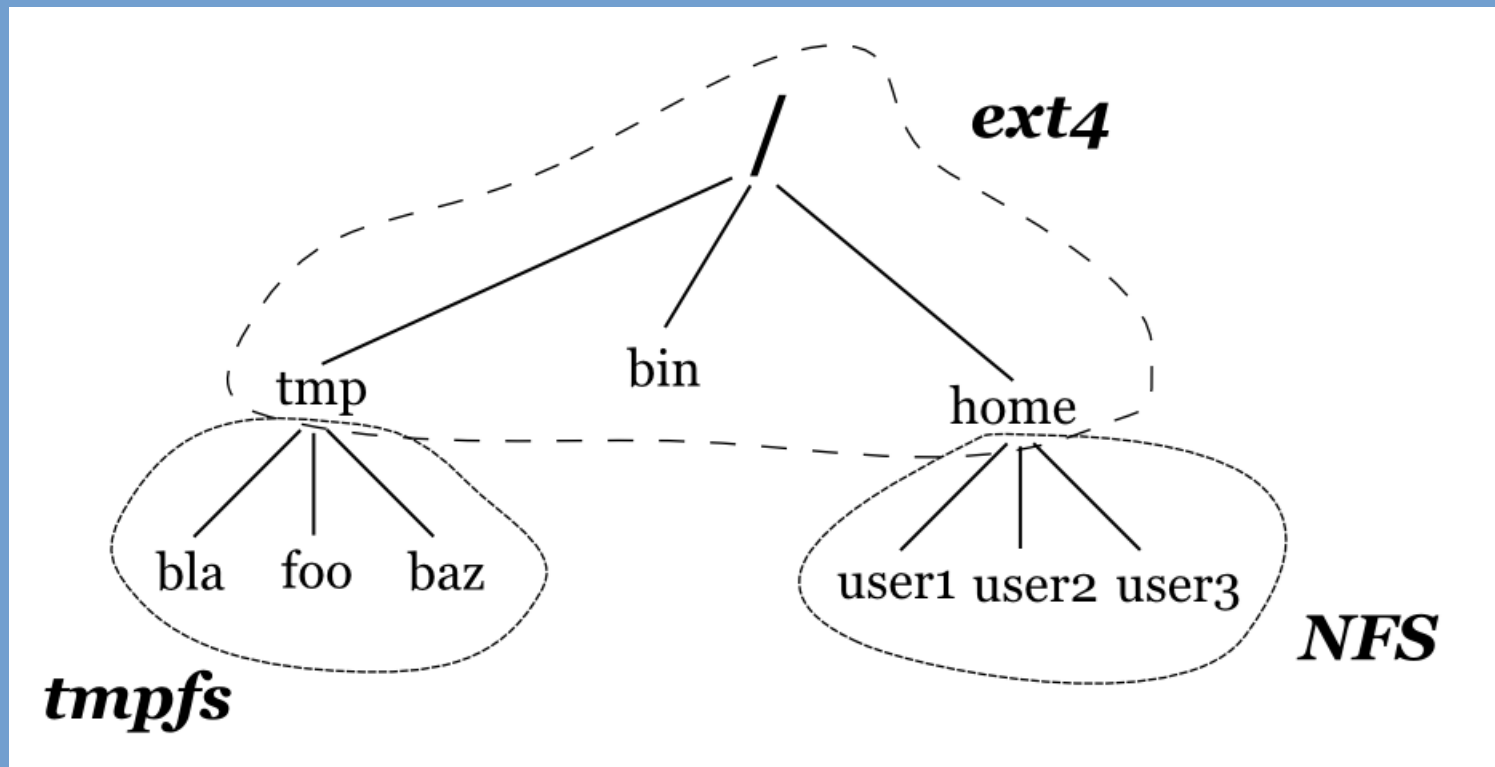
➤ The system calls are programmed against a generic file system interface. Through polymorphism the correct implementation of the call is found at run-time.



Source: Silberschatz et al., Operating System Concepts, 10th Edition

Virtual File System (3)

➤ This allows us to create a single large directory hierarchy in which multiple, different file systems may be attached (mounted):

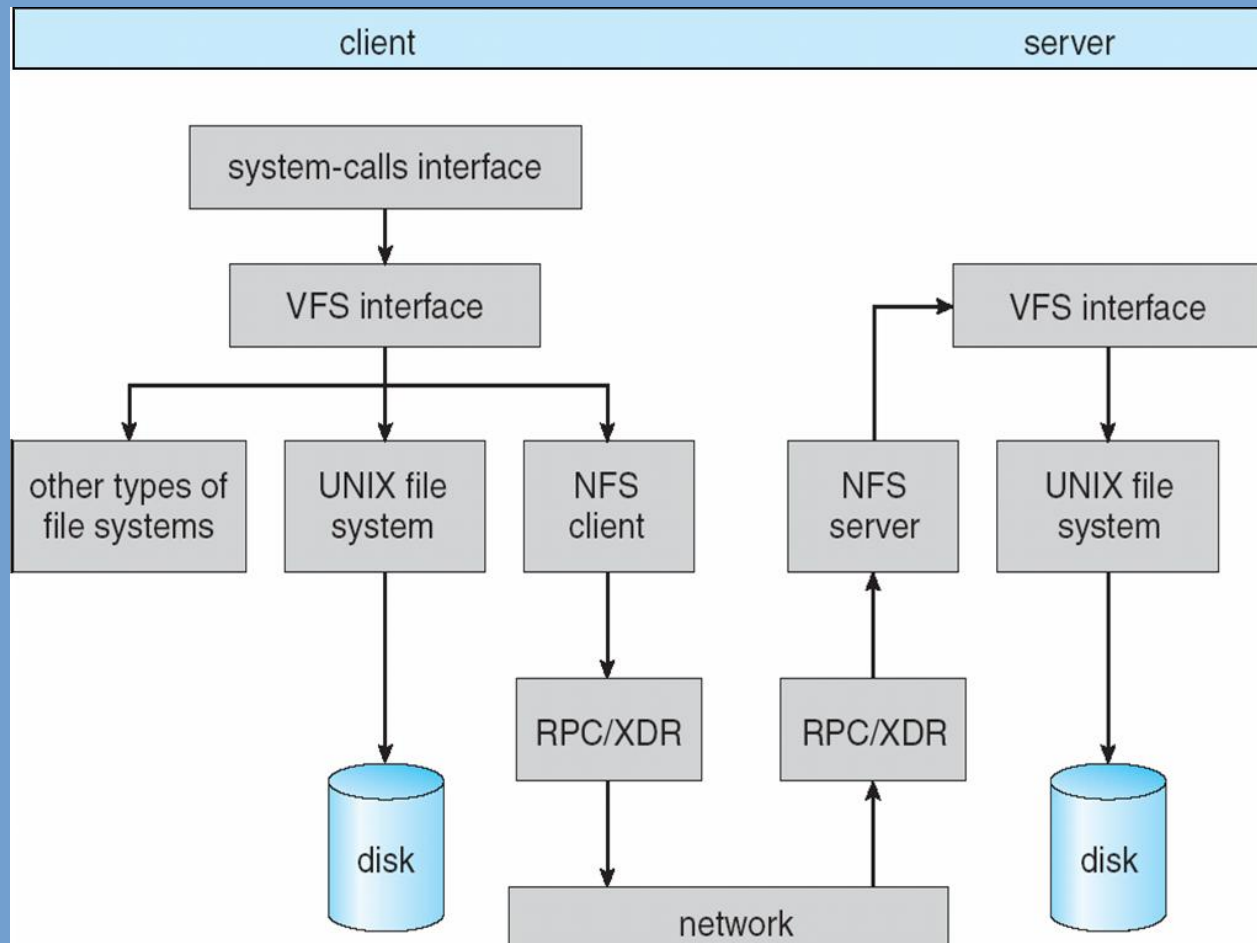


Accessing Files on Remote Systems

- Several schemes are used to access files on remote systems.
- Manual and explicit downloads from servers: FTP and HTTP.
- Cloud storage systems (DropBox, Google Drive).
- Client-server network file systems, such as UNIX NFS and Windows CIFS.
- NFS allows a remote file system to be mounted at a local mount point. After that it can be accessed like a local directory.
- Important: user IDs are stored in the file attributes. This implies that the user and group IDs must be the same on clients and the server.
- To accomplish this, distributed naming services such as LDAP, ActiveDirectory, Novell NDS (and in the past NIS) are used.

The magic of abstraction ...

> We can now understand (and appreciate) this schematic overview of the implementation of a client-server NFS system:



Source: Silberschatz et al., Operating System Concepts, 10th Edition

reld kennen